

Reviewer Pack — Minimal Symplectic Geometry Rank and Communication Matrix Ra...

Entry 9ce4662dac50 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 12:40:20 UTC

Minimal Symplectic Geometry Rank and Communication Matrix Rank Correlation

Entry ID: 9ce4662dac50 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 12:40:20 UTC

1. Conjecture

Field A (mathematical branch): Symplectic Geometry **Field B** (complexity object): Communication Complexity

Statement:

For every Boolean function f with communication complexity rank $r(f)$, the minimal symplectic geometry rank ($\min_rank(G_f)$) of its associated hyperplane arrangement is linearly correlated with $r(f)$, such that $\min_rank(G_f) = \Theta(r(f))$. Equivalently, for any hyperplane arrangement G_f , if $r(f) = O(n^\alpha)$, then $\min_rank(G_f) = O(n^\beta)$ for some $\alpha \leq \beta < 2$.

Rationale (proposer's reasoning):

Symplectic geometry has not been extensively applied to study communication complexity. However, the rank of a hyperplane arrangement can be computed algorithmically and might capture geometric properties related to the complexity of computing f . A correlation between these two quantities could expose new structural insights into communication complexity.

Taxonomy category: symplectic_geometry_communication_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 1b204ab9ba8988c9

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, across at least 24 out of 30 random seeds, $\min_rank(G_f)$ is within a factor of 2 from $r(f)$, measured by the ratio $|\min_rank(G_f)/r(f)| \leq 2$ for all generated Boolean functions f .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "symplectic geometry" AND "communication complexity" AND rank correlation" - "minimal symplectic geometry rank" AND communication matrix rank AND $\theta(n^\alpha)$ - "Boolean function f" AND associated hyperplane arrangement G_f AND linearly correlated with $r(f)$

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=-9, elapsed=198.6s

5.1 Generated Python source

```
import random
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def communication_matrix_rank(f):
        n = len(f)
        matrix = []
        for i in range(2**n):
            row = []
            for j in range(n):
                row.append((i >> j) & 1)
            matrix.append(row)
        rank = 0
        for row in matrix:
            if any(row[j] != 0 for j in range(rank)):
                rank += 1
        return rank

    def construct_hyperplane_arrangement(f):
        n = len(f)
        hyperplanes = []
        for i in range(n):
            hyperplanes.append(random.randint(1, 10))
        return hyperplanes
```

```

def min_symplectic_geometry_rank(hyperplanes):
    n = len(hyperplanes)
    rank = 0
    for h in hyperplanes:
        if any(h != 0):
            rank += 1
    return rank

results = []
for _ in range(30):
    n = random.choice([5, 10, 15, 20, 30, 40])
    f = generate_random_boolean_function(n)
    r_f = communication_matrix_rank(f)
    G_f = construct_hyperplane_arrangement(f)
    min_rank_G_f = min_symplectic_geometry_rank(G_f)

    if r_f == 0:
        continue

    ratio = abs(min_rank_G_f / r_f)
    results.append((r_f, min_rank_G_f, ratio))

if not results:
    return {
        "metric_name": "min_symplectic_geometry_rank_over_communication_matrix_rank",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

mean_ratio = sum(ratio for _, _, ratio in results) / len(results)
support_fraction = sum(1 for _, _, ratio in results if 0.5 <= ratio <= 2) / len(results)

return {
    "metric_name": "min_symplectic_geometry_rank_over_communication_matrix_rank",
    "metric_value": mean_ratio,
    "instances_tested": len(results),
    "n_max": max(n for _, _, _ in results),
    "conjecture_holds": support_fraction >= 0.8,
    "counterexample": "" if support_fraction >= 0.8 else f"mean_ratio={mean_ratio}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 7 for i in range(5, 30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_ratio = sum(result["metric_value"] for result in results if result["metric_value"] is not

```

```

support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_ratio} std=0.0 support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_ratio} std=0.0 support_fraction={support_fraction}")
else:
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

(empty)

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means the pre-registered support condition could not be unambiguously met. | next: Re-run the test with a different set of seeds to ensure it completes successfully and verify if the pre-registered support condition is met.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17873
2	preregistration	ollama_remote	glm4:latest	0	0	14768
3	novelty	ollama_remote	glm4:latest	0	0	8775
4	novelty	ollama_remote	glm4:latest	0	0	8772
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15231
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12300
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	98280
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18253
9	judge	ollama_remote	glm4:latest	0	0	87984

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 282237 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in [research/audit/9ce4662dac50.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9ce4662dac50.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/9ce4662dac50.tar.gz` (if generated)